

SPECYFIKACJA PROJEKTU

Federacyjna detekcja anomalii w strumieniu danych GTFS-Realtime transportu publicznego ZTM w warunkach dryfu koncepcji

Projekt akademicki z zakresu uczenia federacyjnego

Źródło danych: Zarząd Transportu Metropolitalnego, platforma Otwarte Dane GZM

Czas realizacji: 4 tygodnie

Wersja dokumentu: 1.2 • Sierpień 2026

Spis treści

Spis treści	2
1. Wstęp i motywacja	4
2. Rozpoznanie: analiza stanu wiedzy.....	4
2.1. Uczenie federacyjne	4
2.2. Detekcja anomalii w szeregach czasowych	5
2.3. Dryf koncepcji i uczenie na strumieniu	5
2.4. Standard GTFS-Realtime	5
3. Rozpoznanie źródła danych: strumień GTFS-RT ZTM	6
3.1. Struktura operatorska jako podział federacyjny	6
3.2. Charakterystyka i ograniczenia danych	7
4. Wykorzystanie istniejącej infrastruktury: projekt gtfs-olap	7
4.1. Mapowanie komponentów gtfs-olap na moduły projektu	7
4.2. Zasób danych historycznych	8
4.3. Decyzje projektowe wynikające z istniejącej infrastruktury	8
5. Pytanie projektowe i hipotezy badawcze.....	9
6. Zakres projektu	9
6.1. W zakresie.....	9
6.2. Poza zakresem, jako kierunki dalszych prac	10
7. Architektura systemu	10
7.1. Moduł A: kolektor	10
7.2. Moduł B: potok cech	10
7.3. Moduł C: symulacja federacji.....	11
7.4. Moduł D: detekcja i adaptacja	11
8. Schemat cech wejściowych modelu	11
8.1. Wejście modelu i błąd rekonstrukcji	12
9. Stos technologiczny i biblioteki	12
9.1. Wymagania sprzętowe	13
10. Zdarzenia syntetyczne, plan eksperymentów i metryki.....	13
10.1. Definicje zdarzeń syntetycznych	13
10.2. Procedura wstrzykiwania.....	14
10.3. Plan eksperymentów	14
10.4. Metryki	15
11. Bezpieczeństwo federacji i testy odporności	15
11.1. Badane klasy ataków	15

11.2. Badane mechanizmy obronne	15
11.3. Plan testów odporności	16
12. Harmonogram realizacji	16
13. Analiza ryzyk	17
14. Oczekiwane rezultaty i kryteria ukończenia	17
14.1. Kod i środowisko	17
14.2. Dane	18
14.3. Wyniki eksperymentów	18
14.4. Raport i prezentacja	18
14.5. Kryteria ukończenia.....	18
15. Bibliografia i źródła	19

1. Wstęp i motywacja

Współczesne systemy transportu publicznego generują nieprzerwane strumienie danych telemetrycznych, obejmujące pozycje pojazdów oraz odchylenia od rozkładu jazdy. Zarząd Transportu Metropolitalnego, organizujący jeden z największych systemów komunikacji miejskiej w Europie, udostępnia te dane publicznie w otwartym standardzie GTFS-Realtime. Przewozy na zlecenie ZTM realizuje wielu niezależnych operatorów. Ruch tramwajowy w całości obsługuje spółka Tramwaje Śląskie, dysponująca siecią ponad 300 km torowisk. Przewozy autobusowe wykonują miejskie przedsiębiorstwa komunikacyjne, między innymi PKM Katowice, PKM Gliwice, PKM Sosnowiec i PKM Tychy, a uzupełniają je mniejsi przewoźnicy prywatni, tacy jak Transgór Mysłowice. Taka struktura tworzy naturalnie rozproszony, niejednorodny układ danych.

Układ ten stanowi modelowy przypadek użycia dla uczenia federacyjnego, czyli paradygmatu, w którym wiele podmiotów wspólnie trenuje model uczenia maszynowego bez wymiany surowych danych. W realnym scenariuszu operatorzy transportu mogliby nie chcieć lub nie móc udostępniać sobie nawzajem szczegółowej telemetrii pojazdów, na przykład ze względów kontraktowych czy konkurencyjnych, a mimo to każdy z nich skorzystałby na wspólnym modelu wykrywającym zakłócenia w sieci. Uczenie federacyjne rozwiązuje dokładnie ten problem: lokalne dane pozostają u klienta, a wymieniane są wyłącznie parametry modelu.

Celem projektu jest zaprojektowanie, implementacja i weryfikacja eksperymentalna systemu federacyjnej detekcji anomalii działającego na strumieniu danych GTFS-RT. Przez anomalie rozumiane są nietypowe zakłócenia ruchu, takie jak nagłe opóźnienia obejmujące wiele kursów, zanik części pojazdów z sieci czy załamanie punktualności. System uwzględnia dryf koncepcji, czyli trwałe zmiany rozkładu danych wynikające na przykład ze zmian rozkładów jazdy, remontów lub sezonowości. Kluczowym problemem badawczym projektu jest rozróżnienie anomalii, którą system powinien sygnalizować, od dryfu, do którego system powinien się zaadaptować, oraz zbadanie, czy federacyjna agregacja wiedzy poprawia jakość detekcji względem modeli czysto lokalnych.

Projekt jest realizowany w reżimie czasowym jednego miesiąca, dlatego zakres został zawężony do rdzenia badawczego. Naturalne dryfy sezonowe, trudne do zaobserwowania w tak krótkim oknie, uzupełniono kontrolowanymi dryfami syntetycznymi wstrzykiwanymi do zarchiwizowanego strumienia, co zapewnia pełną znajomość etykiet zdarzeń. Istotnym atutem projektu jest oparcie się na istniejącej infrastrukturze: zrealizowanej wcześniej przez autora hurtowni danych gtfs-olap, opisanej w rozdziale 4, która obejmuje działający potok ETL dla tego samego strumienia GTFS-RT oraz około miesiąc zarchiwizowanych danych historycznych. Dzięki temu ciężar prac przenosi się z inżynierii danych na część badawczą, obejmującą uczenie federacyjne, detekcję i eksperymenty.

2. Rozpoznanie: analiza stanu wiedzy

2.1. Uczenie federacyjne

Uczenie federacyjne zostało spopularyzowane przez algorytm FedAvg, zaproponowany przez McMahanę i współautorów w 2017 roku. Serwer centralny działa w nim iteracyjnie: rozsyła bieżący model globalny do wybranych klientów, każdy klient wykonuje lokalny trening na własnych danych, a następnie serwer agreguje otrzymane wagi średnią ważoną liczebnością lokalnych zbiorów. Fundamentalnym wyzwaniem tego paradygmatu jest niejednorodność rozkładów danych między klientami, która spowalnia zbieżność i pogarsza jakość modelu globalnego. W projekcie problem ten występuje naturalnie: rozkład opóźnień linii tramwajowej w centrum Katowic różni się zasadniczo od podmiejskiej linii autobusowej.

Algorytm FedProx, opisany przez Li i współautorów w 2020 roku, łagodzi skutki niejednorodności przez dodanie do lokalnej funkcji straty składnika regularizacyjnego, który karze nadmierne oddalanie się wag lokalnych od modelu globalnego. FedProx zostanie użyty w projekcie jako druga strategia agregacji, porównywana z FedAvg. Bardziej rozbudowane podejścia, takie jak SCAFFOLD, personalizacja czy klastrowanie klientów, pozostają poza zakresem miesięcznym i zostały opisane jako kierunki dalszych prac.

2.2. Detekcja anomalii w szeregach czasowych

Standardowym nienadzorowanym podejściem do detekcji anomalii w danych sekwencyjnych jest autoenkoder, czyli sieć neuronowa trenowana do rekonstrukcji danych typowych, wolnych od anomalii. Próbkę odbiegającą od nauczonego wzorca są rekonstruowane z dużym błędem, a błąd rekonstrukcji pełni rolę wskaźnika anomalności. Dla danych sekwencyjnych stosuje się warianty rekurencyjne, w szczególności autoenkoder oparty na komórkach LSTM, opisany przez Malhotrę i współautorów w 2016 roku, lub lżejszy wariant oparty na komórkach GRU. Model koduje okno czasowe cech do reprezentacji ukrytej i odtwarza je z powrotem, a próg alarmu wyznacza się jako wysoki percentyl rozkładu błędów rekonstrukcji na danych walidacyjnych. Termin autoenkoder jest przyjętym w polskiej literaturze określeniem tej architektury i będzie stosowany konsekwentnie.

W wariacie federacyjnym każdy klient trenuje autoenkoder lokalnie na własnym strumieniu, a agregacja wag buduje wspólną reprezentację typowego ruchu w całej sieci. Hipoteza motywująca projekt brzmi: model federacyjny powinien wykrywać u klienta C taki typ anomalii, który w danych treningowych widzieli wyłącznie klienci A i B. Zjawisko to, nazywane dalej transferem wiedzy o anomaliiach, stanowi przedmiot centralnego eksperymentu projektu E3, opisanego w rozdziale 10.

2.3. Dryf koncepcji i uczenie na strumieniu

Dryf koncepcji to zachodząca w czasie zmiana rozkładu danych lub zależności między cechami a celem predykcji. W transporcie publicznym dryfy są codziennością: korekty rozkładów jazdy, objazdy remontowe, sezonowość roku szkolnego i wakacji, wymiany taboru. Model detekcji anomalii, który się nie adaptuje, po każdej takiej zmianie zaczyna generować lawinę fałszywych alarmów. Model adaptujący się bezkrytycznie nauczy się z kolei przewlekłego incydentu jako nowego wzorca i przestanie go sygnalizować. Sednem logiki decyzyjnej systemu jest więc rozstrzygnięcie: krótkie odchylenie oznacza anomalie, a trwałe przesunięcie rozkładu oznacza dryf.

Klasycznym detektorem dryfu jest algorytm adaptacyjnego okna ADWIN, opisany przez Bifetą i Gavalde w 2007 roku. Utrzymuje on okno o zmiennej długości nad monitorowaną statystyką, którą w projekcie jest strumień błędów rekonstrukcji autoenkodera. Okno dzielone jest na dwa podokna: pierwsze obejmuje obserwacje starsze, z początku okna, a drugie obserwacje najnowsze. Jeżeli różnica średnich obu podokien przekroczy próg wyznaczony nierównością Hoeffdinga, algorytm sygnalizuje dryf i odrzuca obserwacje starsze, uznając je za pochodzące sprzed zmiany. ADWIN jest dostępny w bibliotece River, działa przyrostowo na strumieniu i jest tani obliczeniowo, co czyni go naturalnym wyborem dla projektu.

2.4. Standard GTFS-Realtime

GTFS-Realtime to otwarty standard publikacji danych o bieżącym stanie sieci transportowej, komplementarny do statycznego GTFS obejmującego rozkłady, przystanki i trasy. Dane serializowane są w formie Protocol Buffers. Korzeniem schematu jest komunikat FeedMessage zawierający nagłówek i listę encji. Specyfikacja przewiduje trzy typy encji: TripUpdate z przewidywanymi i rzeczywistymi opóźnieniami kursów na przystankach, VehiclePosition z pozycją pojazdu i jego przypisaniem do kursu oraz Alert z

komunikatami o zakłóceniach. W strumieniu ZTM encje Alert nie są w praktyce wykorzystywane, dlatego projekt nie opiera na nich żadnego elementu ewaluacji. Strumień GTFS-RT jest ulotny: zawiera wyłącznie stan bieżący i jest nadpisywany, dlatego warunkiem prowadzenia eksperymentów jest własne archiwum budowane przez cykliczne odpytywanie.

3. Rozpoznanie źródła danych: strumień GTFS-RT ZTM

Dane GTFS-RT dla sieci ZTM publikowane są na platformie Otwarte Dane GZM pod adresem otwartedane.metropoliagzm.pl, na licencji Creative Commons Attribution, która jednoznacznie dopuszcza wykorzystanie akademickie z podaniem źródła. Zasoby zbioru danych o nazwie Rozkłady jazdy w formacie GTFS RT, w wersji specyfikacji 2.0, zestawiono poniżej.

Zasób	URL	Zawartość
tripUpdates	https://gtfsrt.transportgzm.pl:5443/gtfsrt/gzm/tripUpdates	Opóźnienia kursów na przystankach
vehiclePositions	https://gtfsrt.transportgzm.pl:5443/gtfsrt/gzm/vehiclePositions	Pozycje pojazdów w czasie rzeczywistym
all	https://gtfsrt.transportgzm.pl:5443/gtfsrt/gzm/all	Wszystkie encje w jednym strumieniu
gzm / info	https://gtfsrt.transportgzm.pl:5443/gtfsrt/gzm	Zasób główny i metadane strumienia

Komplementarnie ZTM publikuje na tej samej platformie statyczny GTFS w wersji rozszerzonej, zawierający między innymi pliki routes.txt, trips.txt, stops.txt, stop_times.txt i agency.txt. Stacyjny GTFS jest niezbędny do mapowania identyfikatorów kursów ze strumienia na linie i operatorów, do wyznaczenia rozkładowych czasów przejazdu jako punktu odniesienia dla opóźnień oraz do podziału danych na klientów federacji według naturalnych granic organizacyjnych.

3.1. Struktura operatorska jako podział federacyjny

Przewozy na zlecenie ZTM wykonują trzy grupy podmiotów. Całość ruchu tramwajowego obsługuje spółka Tramwaje Śląskie. Trzon przewozów autobusowych realizują miejskie przedsiębiorstwa komunikacyjne, w tym PKM Katowice obsługujące około 59 linii w Katowicach, Chorzowie, Siemianowicach Śląskich, Bytomiu i Mysłowicach, a także PKM Gliwice, PKM Sosnowiec i PKM Tychy. Uzupełnieniem są mniejsi przewoźnicy prywatni wyłaniani w przetargach. Podział na klientów federacji zostanie przeprowadzony według operatora; mapowanie kursu na operatora jest kompletne i dostępne w wymiarze dim_operator z projektu gtfs-olap, opisanego w rozdziale 4. Docelowa liczba klientów wynosi od 6 do 10, przy czym najmniej

przewoźnicy zostaną w razie potrzeby połączeni w jednego klienta zbiorczego. Taki podział gwarantuje autentyczną niejednorodność danych: klient tramwajowy ma inną charakterystykę opóźnień i odstępów między kursami niż klient autobusowy podmiejski.

3.2. Charakterystyka i ograniczenia danych

Poniższe własności strumienia ZTM zostały zweryfikowane empirycznie w trakcie realizacji projektu gtfs-olap i stanowią wiedzę zastaną, a nie ryzyko do zbadania.

- Odpytywanie strumienia co 20 sekund jest stabilnym ustawieniem roboczym. Strumień miewa jednak awarie po stronie ZTM, obserwowane średnio raz na dwa tygodnie; kolektor rejestruje wtedy przerwę, której nie da się uzupełnić.
- Encje TripUpdate publikowane są bez identyfikatora przystanku. Dopasowanie do rozkładu wymaga wyszukiwania po parze złożonej z identyfikatora kursu i numeru sekwencyjnego zatrzymania; mechanizm ten jest zaimplementowany i sprawdzony w istniejącym potoku ETL.
- ZTM nie raportuje wprost kursów niezrealizowanych. Brak realizacji objawia się wyłącznie zanikiem obserwacji danego kursu ze strumienia, co ma bezpośrednie konsekwencje dla definicji cech w rozdziale 8.
- Encje Alert nie są w praktyce publikowane, dlatego ewaluacja opiera się w całości na zdarzeniach syntetycznych z pełną znajomością etykiet.

4. Wykorzystanie istniejącej infrastruktury: projekt gtfs-olap

Autor zrealizował uprzednio projekt gtfs-olap, dostępny w repozytorium github.com/KsarkesL9/gtfs-olap. Jest to hurtownia danych z potokiem ETL dla tego samego źródła, czyli strumienia GTFS-RT ZTM. System obejmuje potok pobierania encji TripUpdate z odpytywaniem co 20 sekund, ponawianiem prób z wykładniczym oczekiwaniem i pamięcią podręczną rozkładu, potok zasilania słowników ze statycznego GTFS pobieranego z interfejsu CKAN platformy Otwarte Dane GZM, magazyn w bazie PostgreSQL z rozszerzeniem TimescaleDB wykorzystujący hipertabele i agregacje ciągłe, model gwiazdy z tabelami wymiarów oraz warstwę prezentacji w Power BI i Kepler.gl. Niniejszy projekt świadomie buduje na tej infrastrukturze, co zasadniczo zmienia jego profil ryzyka i harmonogram.

4.1. Mapowanie komponentów gtfs-olap na moduły projektu

Komponent gtfs-olap	Rola w niniejszym projekcie	Status
Potok ETL strumienia: pętla pobierania TripUpdates, ponawianie prób, pamięć podręczna rozkładu	Moduł A, czyli kolektor: adaptacja zamiast budowy od zera	gotowe, drobne rozszerzenia
Potok ETL statycznego GTFS: słowniki i wyszukiwanie rozkładowe	Mapowanie kursu na linię i operatora; czasy rozkładowe	gotowe
Hipertabela fakt_opoznienia	Archiwum obserwacji opóźnień na poziomie pojedynczego zatrzymania	gotowe
Agregacja ciągła ca_opoznienia_15min grupowana po linii i operatorze	Podstawa wektora cech na okno 15 minut na klienta, rozdział 8	gotowe

Komponent gtfs-olap	Rola w niniejszym projekcie	Status
Wymiary dim_operator i dim_linia	Naturalny podział danych na klientów federacji	gotowe
Wymiar dim_wersja_rozkładu	Rzeczywiste etykiety momentów dryfu: każda zmiana wersji rozkładu jazdy to oznaczony kandydat na dryf	gotowe, wartość dodana
Hipertabela fakt_etl_run	Rejestr przebiegów kolektora, w tym inwentaryzacja przerw w zbieraniu	gotowe
Zbiór historyczny: około miesiąc obserwacji	Natychmiastowo dostępny zbiór treningowy	gotowe, kluczowy zasób

Wątpliwość może budzić oparcie cech na agregacji ciągłej zamiast na surowych faktach. Agregacje ciągłe TimescaleDB są jednak wyliczane deterministycznie z hipertabeli faktów i odświeżane cykliczną polityką, a okno 15 minut stanowi docelową granulację cech modelu, więc agregacja nie powoduje utraty informacji istotnej dla detekcji. Surowe fakty pozostają przy tym dostępne i umożliwiają przeliczenie cech w innej granulacji, gdyby analiza wrażliwości w tygodniu trzecim tego wymagała.

4.2. Zasób danych historycznych

Baza z projektu gtfs-olap zawiera około miesiąc zarchiwizowanych obserwacji opóźnień. Zbiór treningowy istnieje zatem od pierwszego dnia projektu, więc modelowanie i eksperymenty federacyjne startują natychmiast. Nowo zbierane dane, gromadzone równolegle przez cały okres realizacji, pełnią rolę zbioru testowego rozłącznego czasowo ze zbiorem treningowym. Dane historyczne zawierają luki wynikające z awarii strumienia po stronie ZTM. Pierwszym zadaniem projektu jest więc inwentaryzacja tych luk na podstawie rejestru fakt_etl_run oraz przyjęcie zasad ich obsługi: trening prowadzony jest na ciągłych podzbiorach, a sekwencje wejściowe przecinające lukę są odrzucane.

4.3. Decyzje projektowe wynikające z istniejącej infrastruktury

- D1, cechy wyłącznie z TripUpdates:** gtfs-olap nie zbiera strumienia pozycji pojazdów, a dopisywanie go kosztowałoby kilka dni bez gwarancji jakości pól; w strumieniu ZTM pole prędkości nie jest zresztą publikowane. Projekt rezygnuje więc z cech prędkościowych na rzecz zestawu cech opóźnieniowych i odstępowych opisanego w rozdziale 8. Rozszerzenie o strumień pozycji pozostaje kierunkiem dalszych prac.
- D2, dopisanie surowego archiwum:** do istniejącej pętli ETL zostanie dodany zapis surowych odpowiedzi strumienia na dysk. Koszt implementacji jest pomijalny, a zapis zabezpiecza możliwość ponownego przetworzenia danych przy zmianie potoku.
- D3, format Parquet jako interfejs między hurtownią a częścią uczeniową:** cechy eksportowane są z widoków TimescaleDB do plików Parquet jednorazowym zapytaniem. Część uczeniowa, obejmująca biblioteki Flower, PyTorch i River, pracuje wyłącznie na plikach, bez zależności od działającej bazy, co upraszcza odtwarzanie eksperymentów.
- D4, okno bazowe 15 minut:** przyjęto okno zgodne z istniejącą agregacją ca_opoznienia_15min, eliminując potrzebę ponownej agregacji danych historycznych. Sekwencja wejściowa modelu obejmuje 8 kolejnych okien, czyli 2 godziny kontekstu.

5. **D5, odwrócony harmonogram:** modelowanie rozpoczyna się w tygodniu pierwszym na danych historycznych, a nowe dane zasilają ewaluację w tygodniach trzecim i czwartym, zgodnie z rozdziałem 12.

5. Pytanie projektowe i hipotezy badawcze

Główne pytanie projektowe: czy federacyjnie trenowany model detekcji anomalii, wyposażony w mechanizm rozróżniania anomalii od dryfu koncepcji, osiąga na rzeczywistym strumieniu danych transportu publicznego ZTM lepszą jakość detekcji niż modele czysto lokalne, przy zachowaniu separacji surowych danych między klientami?

Pytanie główne dekomponuje się na następujące hipotezy weryfikowane eksperymentalnie.

1. **H1, transfer wiedzy o anomaliiach:** model federacyjny wykrywa u danego klienta typy anomalii, które w fazie treningu występowały wyłącznie u innych klientów, istotnie lepiej niż model trenowany wyłącznie lokalnie.
2. **H2, adaptacja do dryfu:** system z detekcją dryfu i douczaniem po jego wykryciu generuje istotnie mniej fałszywych alarmów po trwałej zmianie rozkładu danych niż system statyczny, nie tracąc przy tym czułości na anomalie.
3. **H3, koszt federacji:** jakość detekcji modelu federacyjnego jest zbliżona do górnego ograniczenia, jakim jest model scentralizowany trenowany na połączonych danych wszystkich klientów, przy czym FedProx przewyższa FedAvg w warunkach silnej niejednorodności.
4. **H4, odporność federacji:** agregacja odporna na wartości odstające ogranicza wpływ pojedynczego złośliwego klienta na jakość globalnego modelu detekcji w stopniu istotnie większym niż standardowa agregacja uśredniająca.

Pytania pomocnicze: P1, jakie cechy wyznaczone ze strumienia najlepiej charakteryzują typowy ruch; P2, jak dobrać próg alarmu i parametry detektora dryfu, aby zbalansować opóźnienie detekcji i odsetek fałszywych alarmów; P3, jaki jest narzut komunikacyjny federacji wyrażony w megabajtach na rundę.

6. Zakres projektu

6.1. W zakresie

- Adaptacja kolektora z gtfs-olap: dopisanie surowego archiwum zgodnie z decyzją D2 oraz utrzymanie nieprzerwanego zbierania przez cały okres realizacji, wraz z inwentaryzacją luk w danych historycznych.
- Potok inżynierii cech: eksport agregacji 15-minutowych z TimescaleDB do plików Parquet, osobno dla każdego klienta.
- Model detekcji anomalii: autoenkoder rekurencyjny oparty na komórkach GRU lub LSTM, w jednej architekturze dobranej wstępnie, bez rozległego strojenia.
- Symulacja uczenia federacyjnego we frameworku Flower: od 6 do 10 klientów, strategię FedAvg i FedProx.
- Detektor dryfu ADWIN na strumieniu błędów rekonstrukcji, reguła decyzyjna rozróżniająca anomalie od dryfu oraz douczanie modelu po wykryciu dryfu.

- Generator anomalii i dryfów syntetycznych wstrzykiwanych do odtwarzanego archiwum, z pełną znajomością etykiet; dodatkowo analiza jakościowa rzeczywistych zmian wersji rozkładu jako naturalnych kandydatów na dryf.
- Testy odporności federacji na klienta złośliwego wraz z agregacją odporną, zgodnie z rozdziałem 11.
- Ewaluacja porównawcza: model lokalny, federacyjny i scentralizowany; system z adaptacją i bez adaptacji.
- Raport końcowy z wynikami i wnioskami oraz prezentacja.

6.2. Poza zakresem, jako kierunki dalszych prac

- Strumień pozycji pojazdów i cechy prędkościowe, zgodnie z decyzją D1.
- Zaawansowane strategie uczenia federacyjnego: SCAFFOLD, klastrowanie klientów, personalizacja modeli, warianty asynchroniczne.
- Uczenie ustawiczne przeciwdziałające katastrofalnemu zapomnieniu, w tym bufor odtworzeniowy i regularyzacja EWC.
- Prywatność różnicowa i kryptograficznie bezpieczna agregacja.
- Modele grafowe sieci transportowej oraz cechy zewnętrzne, takie jak pogoda i kalendarz wydarzeń.
- Wdrożenie produkcyjne na żywym strumieniu; dopuszczalne wyłącznie jako opcjonalna demonstracja, jeżeli pozwoli czas.

7. Architektura systemu

System składa się z czterech luźno powiązanych modułów. Moduły A i B bazują na infrastrukturze gtfs-olap opisanej w rozdziale 4, a interfejsem między hurtownią a częścią uczeniową są pliki Parquet, zgodnie z decyzją D3. Eksperymenty prowadzone są w trybie odtwarzania zarchiwizowanego strumienia w przyspieszonym czasie, co uniezależnia trening od tempa rzeczywistego strumienia i umożliwia wielokrotne powtarzanie tych samych przebiegów.

7.1. Moduł A: kolektor

- Istniejąca pętla ETL pobiera encje TripUpdate co 20 sekund, ponawia nieudane próby z wykładniczym oczekiwaniem i korzysta z pamięci podręcznej rozkładu z wyszukiwaniem po identyfikatorze kursu i numerze sekwencyjnym zatrzymania. Ta część pozostaje bez zmian i działa nieprzerwanie od pierwszego dnia projektu.
- Rozszerzenie wynikające z decyzji D2: zapis surowych odpowiedzi strumienia na dysk, z partycjonowaniem katalogów według daty i godziny, jako nienaruszalne źródło danych obok bazy.
- Rejestr fakt_etl_run dokumentuje każdy przebieg kolektora. Przerwy w zbieraniu, wynikające z okresowych awarii strumienia po stronie ZTM, są rejestrowane i traktowane jako luki w danych; projekt nie zakłada ich naprawiania, ponieważ utraconych migawek nie da się odtworzyć.

7.2. Moduł B: potok cech

- Źródłem jest agregacja ciągła ca_opoznienia_15min, grupowana po linii i operatorze, mapowana na klientów przez wymiar dim_operator.
- Okna 15-minutowe eksportowane są do plików Parquet osobno dla każdego klienta, a następnie wyliczane są cechy pochodne i sekwencje wejściowe zgodnie z rozdziałem 8.

- Cechy są standaryzowane osobno dla każdego klienta, przy czym statystyki standaryzacji liczone są wyłącznie na danych treningowych danego klienta, zgodnie z zasadą separacji danych w uczeniu federacyjnym.

7.3. Moduł C: symulacja federacji

- Serwer Flower z konfigurowalną strategią agregacji i harmonogramem rund; klienci działają jako procesy symulowane na jednej maszynie.
- Każdy klient wykonuje lokalny trening autoenkodera na własnej partycji danych, walidację lokalną oraz raportuje metryki i wagi do serwera.
- Rejestrowany jest pełny przebieg: strata rekonstrukcji każdego klienta w każdej rundzie oraz rozmiar przesyłanych aktualizacji, stanowiący miarę narzutu komunikacyjnego.

7.4. Moduł D: detekcja i adaptacja

Dla każdego okna sekwencji wejściowej wyznaczany jest błąd rekonstrukcji, zdefiniowany w rozdziale 8. Jeżeli błąd przekracza próg alarmowy klienta, a detektor ADWIN nie sygnalizuje zmiany rozkładu, system zgłasza anomalię. Jeżeli natomiast sygnał ADWIN utrzymuje się przez zadany horyzont, przyjęty wstępnie jako 6 kolejnych okien, system klasyfikuje odchylenie jako dryf. W odpowiedzi na dryf model jest douczany na świeżych danych w krótkiej rundzie federacyjnej, po czym próg alarmowy jest wyznaczany ponownie na zaktualizowanym rozkładzie błędów. Douczanie wyłącznie lokalne, z późniejszą agregacją, stanowi wariant porównawczy badany w eksperymencie E2.

8. Schemat cech wejściowych modelu

Niech k oznacza klienta federacji, a t kolejne okno 15-minutowe. Z agregacji $ca_opoznienia_15min$, po zsumowaniu z poziomu linii do poziomu klienta, dostępne są wielkości pierwotne: liczba obserwacji $n(t)$, czyli liczba raportowanych zatrzymań; suma opóźnień $S(t)$ w sekundach; liczba zatrzymań punktualnych $q(t)$, przy czym za punktualne uznaje się opóźnienie od minus 30 do 60 sekund; wartości skrajne opóźnienia w oknie oraz liczba zatrzymań pominiętych $u(t)$. Zgodnie z charakterystyką strumienia opisaną w rozdziale 3.2 kursy niezrealizowane nie są raportowane wprost, dlatego ich efektem jest spadek $n(t)$, a kategoria pominięć obejmuje zatrzymania jawnie oznaczone w strumieniu jako pominięte.

Na tej podstawie definiowane są cechy wejściowe modelu. Wzory podano w notacji tekstowej.

Cecha	Definicja	Oznaczenie
Średnie opóźnienie	$d(t) = S(t) / n(t)$, w sekundach	$d(t)$
Wskaźnik punktualności	$w(t) = q(t) / n(t)$	$w(t)$
Opóźnienie maksymalne	Największe opóźnienie zarejestrowane w oknie	$d_{max}(t)$
Natężenie obserwacji	$n(t)$ po standaryzacji; spadek wartości odpowiada zanikowi kursów ze strumienia	$n(t)$
Odsetek pominięć	$p(t) = u(t) / n(t)$	$p(t)$
Dynamika opóźnienia	$\Delta d(t) = d(t) - d(t-1)$, czyli przyrost średniego opóźnienia między kolejnymi oknami	$\Delta d(t)$

Cecha	Definicja	Oznaczenie
Odchylenie od profilu dobowego	$r(t) = d(t) - m(g(t), c(t))$, gdzie m to mediana d wyznaczona na danych treningowych osobno dla każdej godziny doby g i typu dnia c , rozróżniającego dzień roboczy od weekendu	$r(t)$
Nieregularność odstępów między kursami	$h(t)$: odchylenie standardowe rzeczywistych odstępów między kolejnymi przyjazdami kursów tej samej linii, wyznaczanych jako czas rozkładowy powiększony o opóźnienie, uśrednione po liniach klienta	$h(t)$
Kodowanie czasu	$\sin$ i $\cos$ godziny doby oraz binarny wskaźnik dnia roboczego	cechy zegara

Wielkości pochodne, czyli $\Delta d(t)$, $r(t)$ oraz $h(t)$, wyliczane są z wielkości pierwotnych po ewentualnym wstrzyknięciu zdarzeń syntetycznych, dzięki czemu zdarzenia propagują się spójnie na cały wektor cech. Kodowanie czasu oraz odchylenie od profilu pozwalają modelowi odróżnić naturalny szczyt poranny od anomalii bez trenowania osobnych modeli dla poszczególnych pór dnia.

8.1. Wejście modelu i błąd rekonstrukcji

Wejściem autoenkodera jest macierz $X(t)$ złożona z wektorów cech ośmiu kolejnych okien, od $t-7$ do t , co odpowiada dwóm godzinom kontekstu. Model odtwarza wejście, a błąd rekonstrukcji $e(t)$ definiowany jest jako średni błąd kwadratowy między $X(t)$ a rekonstrukcją, uśredniony po wszystkich cechach i oknach sekwencji. Błąd $e(t)$ jest więc pojedynczą liczbą nieujemną przypisaną każdemu oknu. Próg alarmowy klienta k wyznaczany jest jako percentyl 99,5 rozkładu wartości $e(t)$ na wydzielonym zbiorze walidacyjnym tego klienta, a jego wartość podlega ponownemu wyznaczeniu po każdej adaptacji do dryfu. Wybór percentyla należy do parametrów badanych w analizie wrażliwości E4.

9. Stos technologiczny i biblioteki

Dobór technologii kierował się trzema kryteriami: dojrzałością i jakością dokumentacji, minimalnym czasem wdrożenia w czterotygodniowym reżimie oraz zgodnością z istniejącą infrastrukturą gtfs-olap. Wszystkie komponenty są otwartoźródłowe.

Technologia	Warstwa	Rola i uzasadnienie
Python 3.11 lub nowszy	język	Standard w uczeniu maszynowym; wspólny język istniejącego potoku ETL i nowej części uczeniowej.
PostgreSQL z TimescaleDB	magazyn	Przejęte z gtfs-olap: hipertabele fakt_opoznienia i fakt_etl_run oraz agregacje ciągłe jako źródło cech; sterownik pycopg.
httpx, protobuf, loguru	dane	Przejęte z gtfs-olap: pobieranie strumienia z ponawianiem prób, dekodowanie binarne, dziennik zdarzeń.
gtfs-realtime-bindings	dane	Oficjalne klasy Pythona utrzymywane przez MobilityData, wygenerowane ze specyfikacji GTFS-RT; wykorzystywane już w istniejącym potoku ETL.

Technologia	Warstwa	Rola i uzasadnienie
PyTorch	model	Implementacja autoenkodera rekurencyjnego; dojrzałe wsparcie sekwencji i pierwszorzędna integracja z frameworkiem Flower.
Flower	federacja	Wiodący framework badawczy uczenia federacyjnego: niezależny od biblioteki uczącej, z trybem symulacji wielu klientów na jednej maszynie, gotowymi strategiami FedAvg i FedProx oraz agregacjami odpornymi; aktywnie rozwijany, seria wydań 1.29 do 1.32 w 2026 roku.
River	dryf	Biblioteka uczenia na strumieniach danych; dostarcza przyrostową, tanią obliczeniowo implementację algorytmu ADWIN, stosowaną do monitorowania błędów rekonstrukcji.
pandas i PyArrow	przetwarzanie	Eksport cech z TimescaleDB do plików Parquet oraz przygotowanie sekwencji treningowych.
NumPy, scikit-learn	ewaluacja	Metryki jakości, krzywe precyzji i czułości, standaryzacja.
Matplotlib	wizualizacja	Wykresy do raportu: krzywe uczenia, opóźnienia detekcji, porównania wariantów.
Docker Compose, Git	inżynieria	Uruchamianie bazy w konfiguracji przejętej z gtfs-olap; wersjonowanie kodu i odtwarzalne środowisko.

9.1. Wymagania sprzętowe

Kolektor działa na dowolnej maszynie pracującej całodobowo, na przykład na niewielkim serwerze wirtualnym lub stałym komputerze domowym; jego obciążenie jest pomijalne. Eksperymenty wymagają komputera z co najmniej 16 GB pamięci operacyjnej. Procesor graficzny jest przydatny, ale niekonieczny, ponieważ autoenkoder rekurencyjny o rozmiarze rzędu dziesiątek lub setek tysięcy parametrów trenuje się w akceptowalnym czasie na procesorze głównym, a symulacja Flower współdzieli jedno urządzenie między klientów.

10. Zdarzenia syntetyczne, plan eksperymentów i metryki

10.1. Definicje zdarzeń syntetycznych

Zdarzenia syntetyczne wstrzykiwane są do odtwarzanego archiwum na poziomie wielkości pierwotnych z rozdziału 8, po czym wielkości pochodne wyliczane są ponownie, dzięki czemu zdarzenie propaguje się spójnie na cały wektor cech. Wszystkie etykiety zdarzeń są znane, co umożliwia dokładny pomiar jakości detekcji. Przyjęto trzy typy anomalii i dwa typy dryfu, zdefiniowane następującymi przekształceniami.

1. **A1, incydent opóźnieniowy:** dla okien t należących do zdarzenia średnie opóźnienie przyjmuje wartość $d(t) + A$, a opóźnienie maksymalne wartość $d_{\max}(t) + A$. Amplituda A przyjmuje wartości 60, 120 lub 300 sekund, a czas trwania zdarzenia wynosi 1, 2 lub 4 okna, czyli od 15 minut do godziny. Wskaźnik punktualności jest przeliczany przy założeniu przesunięcia całego rozkładu opóźnień o A , co dla przyjętych amplitud oznacza spadek $w(t)$ praktycznie do zera.

2. **A2, zanik kursów:** natężenie obserwacji przyjmuje wartość α razy $n(t)$, gdzie współczynnik zachowania α wynosi 0,3, 0,5 lub 0,7. Zgodnie z charakterystyką strumienia zanik kursów nie pozostawia w danych jawnego śladu poza spadkiem liczby obserwacji, dlatego pozostałe wielkości pierwotne pozostają niezmiennione, a odsetek pominięć rośnie wyłącznie wtedy, gdy część zatrzymań zostaje dodatkowo oznaczona jako pominięta, co odwzorowuje wariant mieszany z parametrem 0,1 dodawanym do $p(t)$.
3. **A3, załamanie punktualności:** wskaźnik punktualności przyjmuje wartość γ razy $w(t)$, gdzie γ wynosi 0,3 lub 0,5, przy niezmiennym średnim opóźnieniu; opóźnienie maksymalne rośnie o 180 sekund. Zdarzenie odwzorowuje sytuację, w której część kursów jedzie znacznie za wcześnie, a część znacznie za późno, podczas gdy średnia pozostaje pozornie typowa.
4. **D1, trwałe przesunięcie poziomu opóźnień:** dla wszystkich okien od momentu t_0 do końca strumienia średnie opóźnienie przyjmuje wartość $d(t) + C$, gdzie stała C wynosi 30 lub 60 sekund. Profil dobowy m pozostaje wyznaczony na danych sprzed zmiany, więc odchylenie od profilu $r(t)$ rośnie trwale, co odwzorowuje wejście w życie nowego rozkładu jazdy.
5. **D2, zmiana profilu dobowego:** średnie opóźnienie przyjmuje wartość $d(t)$ razy $1 + \lambda$ razy $s(g(t))$, gdzie s jest znormalizowanym kształtem szczytów komunikacyjnych, a współczynnik λ wynosi 0,3. Zdarzenie odwzorowuje reorganizację częstotliwości kursów zmieniającą względną wysokość szczytu porannego i popołudniowego.

10.2. Procedura wstrzykiwania

Momenty początkowe anomalii losowane są z rozkładu jednostajnego na osi czasu zbioru testowego, z wyłączeniem dwugodzinnych buforów od początku i końca strumienia, od granic luk w danych oraz od innych zdarzeń, tak aby zdarzenia się nie nakładały. Każda konfiguracja typu i parametrów anomalii wstrzykiwana jest dwudziestokrotnie u każdego badanego klienta. Momenty t_0 dryfów wybierane są tak, aby po dryfie pozostawało co najmniej pięć dni strumienia, co umożliwia pomiar przebiegu adaptacji. Całość procedury sterowana jest ziarnem generatora liczb losowych; każdy eksperyment wykonywany jest dla trzech ziaren, a raportowane są średnie wraz z odchyleniami standardowymi.

10.3. Plan eksperymentów

Ozn.	Opis	Weryfikowana hipoteza, oczekiwany wynik
E1	Porównanie wariantów treningu: lokalny, FedAvg, FedProx i scentralizowany, na wspólnym zbiorze testowym z anomaliami A1 do A3	H3; tabela jakości detekcji dla każdego wariantu i klienta
E2	System statyczny wobec adaptacyjnego z detektorem ADWIN i douczaniem, na strumieniu z dryfami D1 i D2 oraz wplecionymi anomaliami	H2; fałszywe alarmy po dryfie, czułość po adaptacji
E3	Transfer wiedzy: anomalia typu A2 obecna w treningu wyłącznie u dwóch klientów, testowana u klienta trzeciego	H1; kluczowy wynik projektu
E4	Analiza wrażliwości: długość sekwencji 4, 8 lub 12 okien, zestaw cech, percentyl progu, częstość rund federacji	P1 i P2; uzasadnienie konfiguracji bazowej

Ozn.	Opis	Weryfikowana hipoteza, oczekiwany wynik
E5	Odporność federacji na klienta złośliwego, zgodnie z planem testów z rozdziału 11	H4; porównanie agregacji uśredniającej z odporną

10.4. Metryki

- **Jakość detekcji:** miara F1 oraz pole pod krzywą precyzji i czułości, liczone na zdarzeniach syntetycznych zarówno na poziomie pojedynczych okien, jak i na poziomie zdarzeń, przy czym zdarzenie uznaje się za wykryte, jeżeli alarm padł w jego trakcie.
- **Opóźnienie detekcji:** czas od początku zdarzenia do pierwszego alarmu, raportowany jako mediana i percentyl 90.
- **Odsetek fałszywych alarmów:** liczba alarmów na dobę poza zdarzeniami, raportowana osobno w oknie doby przed wstrzykniętym dryfem i doby po nim.
- **Metryki federacji:** krzywe zbieżności straty w kolejnych rundach, rozmiar przesyłanych aktualizacji w megabajtach na rundę i klienta, czas rundy.
- **Metryki adaptacji:** czas od momentu dryfu do powrotu odsetka fałszywych alarmów poniżej poziomu sprzed dryfu.

11. Bezpieczeństwo federacji i testy odporności

Uczenie federacyjne poszerza powierzchnię ataku systemu: serwer agregujący przyjmuje aktualizacje wag od uczestników, których danych nie widzi i których uczciwości nie może bezpośrednio zweryfikować. W zastosowaniach przemysłowych jest to problem podstawowy, dlatego projekt obejmuje jawny model zagrożeń oraz plan testów odporności. Przyjęto następujący model zagrożeń: serwer agregujący jest zaufany, natomiast pojedynczy klient federacji może być skompromitowany i działać złośliwie. Kanał komunikacji uznaje się za zabezpieczony transportowo i pozostaje poza zakresem badania.

11.1. Badane klasy ataków

1. **Zatruwanie danych:** złośliwy klient trenuje lokalny model na danych z systematycznie zaniżonymi opóźnieniami, dążąc do tego, aby globalny model uznawał stany incydentalne za typowe i przestał je sygnalizować.
2. **Zatruwanie modelu przez skalowanie aktualizacji:** złośliwy klient mnoży swoją aktualizację wag przez duży współczynnik, w testach równy 10, aby zdominować agregację uśredniającą i przeciągnąć model globalny w wybranym kierunku.
3. **Atak ukierunkowany:** złośliwy klient douczy lokalny model tak, aby rekonstrukcja wzorca odpowiadającego anomalii typu A1 była bezbłędna, przez co globalny model traci czułość na ten konkretny typ zdarzenia u wszystkich klientów.

11.2. Badane mechanizmy obronne

Podstawową obroną jest zastąpienie agregacji uśredniającej agregacją odporną na wartości odstające. W testach porównane zostaną agregacja medianowa po współrzędnych oraz średnia przycinana, odrzucająca skrajne wartości każdej współrzędnej przed uśrednieniem; obie są dostępne lub proste do

zaimplementowania we frameworku Flower. Uzupełniająco badana jest detekcja aktualizacji odstających na podstawie normy różnicy między aktualizacją klienta a bieżącym modelem globalnym, z odrzucaniem aktualizacji przekraczających próg wyznaczony na rozkładzie norm z rund wcześniejszych.

11.3. Plan testów odporności

Testy realizowane są w ramach eksperymentu E5, w konfiguracji z jednym złośliwym klientem na ośmiu uczestników. Miarą skutku ataku jest spadek jakości detekcji anomalii względem przebiegu odniesienia, a miarą skuteczności obrony odsetek tego spadku, który obrona niweluje. Zakres podstawowy obejmuje testy T1 do T4; testy T5 i T6 są opcjonalne i zostaną wykonane, jeżeli pozwoli harmonogram.

- ☐ **T1:** przebieg odniesienia bez ataku, dla agregacji uśredniającej i medianowej; potwierdzenie, że agregacja odporna nie pogarsza istotnie jakości w warunkach uczciwych.
- ☐ **T2:** zatrucie danych jednego klienta przy agregacji uśredniającej; pomiar spadku wykrywalności anomalii u klientów uczciwych.
- ☐ **T3:** powtórzenie testu T2 przy agregacji medianowej i średniej przycinanej; pomiar stopnia zniwelowania skutku ataku.
- ☐ **T4:** skalowanie aktualizacji złośliwego klienta przy obu rodzajach agregacji; pomiar odchylenia modelu globalnego i jakości detekcji.
- ☐ **T5, opcjonalny:** atak ukierunkowany na wzorzec anomalii A1; pomiar wykrywalności tego typu zdarzenia u klientów uczciwych przed atakiem i po nim.
- ☐ **T6, opcjonalny:** wymuszanie fałszywego sygnału dryfu przez klienta raportującego zawyżone błędy rekonstrukcji; ocena odporności reguły decyzyjnej na zbędne douczanie.

12. Harmonogram realizacji

Harmonogram jest odwrócony zgodnie z decyzją D5: dzięki miesięcznemu zbiorowi historycznemu z gtfs-olap modelowanie rozpoczyna się w tygodniu pierwszym, a nowo zbierane dane zasilają ewaluację w tygodniach trzecim i czwartym.

Tydzień	Zadania	Kamień milowy
1	Dzień pierwszy: dopisanie surowego archiwum do działającego potoku ETL, od tego momentu kolektor zbiera nieprzerwanie. Inwentaryzacja luk w danych historycznych na podstawie fakt_etl_run; eksport cech z agregacji do plików Parquet dla każdego klienta; podział na klientów według wymiaru operatora; pierwszy trening autoenkodera lokalnie na danych historycznych; wstępny test integracyjny symulacji Flower.	Model lokalny trenuje na miesiącu danych historycznych; symulacja federacji uruchomiona
2	Trening federacyjny w pełnym obiegu dla strategii FedAvg i FedProx; generator anomalii i dryfów syntetycznych zgodnie z rozdziałem 10; kalibracja progów percentylowych dla każdego klienta.	Pełny obieg od plików Parquet przez trening federacyjny do wyznaczania wskaźnika anomalności; zdarzenia syntetyczne działają
3	Detektor ADWIN, reguła decyzyjna i douczanie po dryfie; wykonanie eksperymentów E1 do E3 na danych	Komplet wyników eksperymentów E1 do E3

Tydzień	Zadania	Kamień milowy
	historycznych; analiza ewentualnych rzeczywistych zmian wersji rozkładu; początek analizy wrażliwości E4.	
4	Dokończenie E4; testy odporności E5 w zakresie T1 do T4; walidacja modeli na nowo zebranych danych, stanowiących zbiór testowy rozłączny czasowo; wykresy i analiza wyników; raport końcowy i prezentacja; bufor co najmniej dwóch dni na opóźnienia. Opcjonalnie demonstracja na żywym strumieniu oraz testy T5 i T6.	Raport, prezentacja i repozytorium

Rozszerzony kolektor startuje pierwszego dnia projektu, ponieważ nowo zbierane dane z trzech do czterech tygodni stanowią zbiór testowy rozłączny czasowo ze zbiorem treningowym. Łącznie projekt dysponuje około dwoma miesiącami obserwacji.

13. Analiza ryzyk

Ryzyko	Waga	Postępowanie
Awarie strumienia po stronie ZTM w okresie zbierania nowych danych, spodziewane średnio raz na dwa tygodnie	wysoka	Przerwy rejestrowane w fakt_etl_run i traktowane jako luki; eksperymenty projektowane na oknach, więc luki nie unieważniają zbioru; trening zabezpieczony danymi historycznymi
Luki w danych historycznych	średnia	Inwentaryzacja w tygodniu pierwszym; trening na ciągłych podzbiorach; odrzucanie sekwencji przecinających lukę
Zmiana wersji rozkładu jazdy w trakcie zbierania nowych danych	średnia	Wersjonowanie przez wymiar dim_wersja_rozkladu; zdarzenie traktowane jako naturalny, oznaczony dryf, stanowiący wartość dodaną analizy
Zbyt krótki okres na wystąpienie naturalnego dryfu	pewna	Założenie projektowe: dryfy syntetyczne z pełną znajomością etykiet, zgodnie z rozdziałem 10, uzupełnione analizą jakościową zmian wersji rozkładu
Opóźnienie względem harmonogramu	średnia	Bufor w tygodniu czwartym; priorytet eksperymentów E1 do E3 przed E4 i E5; testy T5 i T6 oznaczone jako opcjonalne
Niedostateczna moc obliczeniowa	niska	Mały model o rozmiarze dziesiątek lub setek tysięcy parametrów; sekwencyjna symulacja klientów; próbkowanie okien treningowych

14. Oczekiwane rezultaty i kryteria ukończenia

Rezultaty projektu ujęto w formie listy kontrolnej, umożliwiającej bieżące śledzenie postępu i jednoznaczną ocenę kompletności na zakończenie prac.

14.1. Kod i środowisko

- ☐ Rozszerzenie potoku ETL o zapis surowych odpowiedzi strumienia, wraz z partycjonowaniem katalogów.
- ☐ Skrypt inwentaryzacji luk w danych historycznych na podstawie rejestru przebiegów kolektora.
- ☐ Skrypt eksportu cech z agregacji TimescaleDB do plików Parquet, osobno dla każdego klienta.
- ☐ Implementacja autoenkodera rekurencyjnego wraz z procedurą treningu i walidacji.
- ☐ Symulacja federacji we frameworku Flower ze strategiami FedAvg, FedProx oraz agregacjami odpornymi.
- ☐ Moduł detekcji: wyznaczanie błędu rekonstrukcji, progi percentylowe, detektor ADWIN, reguła decyzyjna i douczanie po dryfie.
- ☐ Generator zdarzeń syntetycznych realizujący definicje z rozdziału 10, sterowany ziarnem generatora losowego.
- ☐ Skrypty odtwarzające każdy eksperyment pojedynczym poleceniem, wraz z plikiem blokady wersji zależności.
- ☐ Dokumentacja uruchomienia całości środowiska od zera.

14.2. Dane

- ☐ Zinwentaryzowany zbiór historyczny z opisem luk i pokrycia czasowego.
- ☐ Nowo zebrany zbiór z okresu realizacji, obejmujący surowe odpowiedzi strumienia oraz rekordy w bazie.
- ☐ Wyeksportowane pliki cech dla wszystkich klientów, w wersjach z wstrzykniętymi zdarzeniami i bez nich.
- ☐ Opis zbioru danych: słownik cech, statystyki opisowe, liczność okien dla każdego klienta.

14.3. Wyniki eksperymentów

- ☐ Tabele jakości detekcji dla eksperymentu E1, obejmujące wszystkie warianty treningu i wszystkich klientów, ze średnimi i odchyleniami dla trzech ziaren.
- ☐ Wyniki eksperymentu E2: przebiegi fałszywych alarmów przed dryfem i po nim, czas adaptacji, porównanie systemu statycznego z adaptacyjnym.
- ☐ Wyniki eksperymentu E3 dokumentujące transfer wiedzy o anomaliach między klientami.
- ☐ Wyniki analizy wrażliwości E4 wraz z uzasadnieniem konfiguracji bazowej.
- ☐ Wyniki testów odporności T1 do T4 z eksperymentu E5, z miarami skutku ataku i skuteczności obrony.
- ☐ Wykresy: krzywe zbieżności federacji, rozkłady opóźnienia detekcji, krzywe precyzji i czułości, narzut komunikacyjny.

14.4. Raport i prezentacja

- ☐ Raport końcowy obejmujący specyfikację, metodykę, wyniki, dyskusję ograniczeń i kierunki dalszych prac.
- ☐ Prezentacja z pokazem kluczowych wyników; opcjonalnie demonstracja na żywym strumieniu.

14.5. Kryteria ukończenia

1. **K1:** system federacyjny osiąga jakość detekcji nie gorszą od modeli lokalnych w eksperymencie E1 oraz wykazuje mierzalny transfer wiedzy w eksperymencie E3, co weryfikuje hipotezę H1.
2. **K2:** wariant adaptacyjny redukuje odsetek fałszywych alarmów po dryfie o co najmniej połowę względem wariantu statycznego, przy spadku czułości nieprzekraczającym pięciu punktów procentowych, co weryfikuje hipotezę H2.
3. **K3:** agregacja odporna niweluje co najmniej połowę spadku jakości wywołanego atakami z testów T2 i T4, co weryfikuje hipotezę H4.
4. **K4:** każdy eksperyment daje się odtworzyć pojedynczym poleceniem z zarchiwizowanych danych.

15. Bibliografia i źródła

1. McMahan B. i in., Communication-Efficient Learning of Deep Networks from Decentralized Data, AISTATS 2017.
2. Li T. i in., Federated Optimization in Heterogeneous Networks, MLSys 2020.
3. Bifet A., Gavalda R., Learning from Time-Changing Data with Adaptive Windowing, SIAM SDM 2007.
4. Malhotra P. i in., LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection, ICML Workshop 2016.
5. Blanchard P. i in., Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent, NeurIPS 2017.
6. Yin D. i in., Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates, ICML 2018.
7. Beutel D. J. i in., Flower: A Friendly Federated Learning Research Framework, arXiv:2007.14390; dokumentacja: flower.ai/docs.
8. Montiel J. i in., River: Machine Learning for Streaming Data in Python, JMLR 22, 2021; riverml.xyz.
9. Specyfikacja GTFS-Realtime i klasy językowe: gtfs.org/documentation/realtime, github.com/MobilityData/gtfs-realtime-bindings.
10. Otwarte Dane GZM, zbiór Rozkłady jazdy w formacie GTFS RT: otwartedane.metropoliagzm.pl/dataset/rozklady-jazdy-i-lokalizacja-przystankow-gtfs-rt, licencja CC-BY.
11. Otwarte Dane GZM, zbiór Rozkłady jazdy i lokalizacja przystanków GTFS wersja rozszerzona.
12. Projekt gtfs-olap autora, hurtownia danych i potok ETL GTFS oraz GTFS-RT ZTM: github.com/KsarkesL9/gtfs-olap.